



Elastic meetup

Plural + Elastic

An awesome combo for agentic ops

+

Plural

+

August 2026

A quick background

Observability for the last decade

+

+

01

Scale-out datastores

Metrics, logs, and traces at volume
— the expensive, stateful core.

02

Telemetry collection agents

Ship every signal from every host,
container, and service.

03

WYSIWYG dashboards

Point-and-click charts and alerting
on top of the data.

Bundled together, this is the standard Datadog stack.

A quick background

Ideal State: Disaggregate the stack

Why: obscene vendor pricing power	Weak, complex OSS alternatives left one out-of-the-box option. See your Datadog bill.
What's happening	OTEL handling the collector layer, OSS datastore innovation is handling the data layer
Why it matters at Plural	A single-tenant cloud is prohibitively expensive to run on standard off-the-shelf solutions — so this stack runs our own infrastructure.

What we run

Plural's open source data stack

Metrics	VictoriaMetrics	Scale-out, performant reimplementa-tion of Prometheus.
Logs	Elastic	Scale-out, full-text search. ClickHouse and Loki are also worth a look.
Traces	Grafana Tempo	Not built yet on our side — Tempo is the usual solution.

What we run

The rest of the ecosystem

+

+

Collection

OpenTelemetry

Broad, vendor-neutral telemetry collection. Getting closer to Datadog-agent parity, but not there yet.

vmagent is the more performant option where it counts.

Visualization + alerting

Grafana

Dashboards and alerting across every datastore in the stack.

Much weaker than Datadog — people dealt with it. More on this shortly.

Where agents fit in

01 Maintain the data layer

Elastic and VictoriaMetrics operations are complex and tedious. Agents handle them end to end now.

02 Commoditize the dashboard

The WYSIWYG layer you paid Datadog for becomes generated, on demand.

03 Focus spend on what is truly difficult

The data. Everything above it is getting automated.

Elastic foundations: index automation

Rollover indices, not giant ones

Roll on primary shard size and age so shards stay evenly sized.

Shard-size rules keep the cluster balanced

Balanced shards mean predictable hot-tier load — no manual reindexing.

Automation first, agents second

A well-automated cluster is a surface agents can safely operate.

```
PUT _ilm/policy/logs-rollover
{
  "policy": {
    "phases": {
      "hot": {
        "actions": {
          "rollover": {
            "max_primary_shard_size": "50gb",
            "max_age": "7d"
          }
        }
      }
    },
    "delete": {
      "min_age": "30d",
      "actions": { "delete": {} }
    }
  }
}
```

Cluster definition with ECK

The cluster is one CRD

Node sets, storage, version, and config in a single Kubernetes resource.

The operator does the ops

ECK handles rolling upgrades, scaling, and certificates from the spec.

GitOps-ready by construction

The spec lives in git — the exact surface an agent can edit and PR.

```
apiVersion: elasticsearch.k8s.elastic.co/v1
kind: Elasticsearch
metadata:
  name: logs
spec:
  version: 8.17.0
  nodeSets:
  - name: hot
    count: 3
    volumeClaimTemplates:
    - metadata:
        name: elasticsearch-data
      spec:
        resources:
          requests:
            storage: 1Ti
```


Day-2 Elastic ops with agents

Disk expansion

Most critical

Node set expansion

Horizontal scaling

CPU / RAM expansion

Vertical scaling

+

+

Plural Workbenches cron

Runs on a schedule
against the cluster



Read the signals

Walks ES time-series
metrics to decide which
move is needed



Author the GitOps
change

Edits the ECK spec and
opens the PR



Operator reconciles

ECK applies the
change safely

Getting fancy: http actions can issue ES API calls directly.

Agents for metrics

Agents on the metrics datastore

“How do you get devs on board with a Datadog migration if they end up stuck with ugly Grafana dashboards?”

Same telemetry work, no dashboards

Agentic ops runs the analyses directly — nothing to maintain or watch constantly.

Spend on data, automate the experience

Budget goes to the data layer; resolution and resource optimization get automated on top.

End-to-end analyses with Plural

“Why did p99 latency spike overnight?”

Query metrics and logs, correlate with deploys, name the culprit.

“Which services are over-provisioned?”

Walk utilization time series, author the right-sizing PRs.

“What's driving the error-rate jump?”

Full-text search across Elastic, cluster the failures, propose the fix.



Thanks

plural.sh + github.com/pluralsh + docs.plural.sh